

CIA-3: PROJECT REPORT

Corporate HR and Payroll Management System

Advanced JavaScript Backend Frameworks (Node.js and Express.js)

Project Code: P04

Team Number: 60

Semester: V

Institution: Christ University

Submission date: 10 September 2026 [10-09-2026]

Team Details

S No	Student Name	Roll No	Department	Class
1	Sri Ramanarayana S N	2461029	Department of CSE	5BTIT
2	Srujani V	2461030	Department of CSE	5BTIT
3	Sujitha V	2461031	Department of CSE	5BTIT
4	Theertha J	2461033	Department of CSE	5BTIT

GitHub Repository: https://github.com/SriRam-0511/L-T_Sem5.git

Project Overview

The Corporate HR and Payroll Management System is a role-based web application for centralising routine workforce administration. It gives HR administrators a structured way to maintain employee records, departments, attendance, leave, payroll, holidays, and reports. Employees can access only their personal information and requests, while managers obtain team-level visibility and approval controls.

Problem Statement

Spreadsheets and disconnected attendance records make employee administration slow, difficult to audit, and vulnerable to calculation mistakes. The proposed system creates one consistent source of truth for employee activity and converts attendance and approved leave data into dependable monthly payroll inputs.

Project Objectives

Objective	Expected Outcome
Centralise employee data	One secured employee profile with departmental, designation, manager, and role information.

Automate attendance and leave workflows	Clock events, balances, approvals, and status transitions are recorded with ownership checks.
Support accurate payroll	Monthly salary is calculated from base pay, attendance rules, and leave deductions.
Provide role based access	Employee, Manager, and HR Admin actions are separated by JWT authorisation middleware.
Improve decision making	Attendance and payroll reports summarise departmental trends and costs.

Scope

The minimum project scope covers the thirteen prescribed modules: onboarding and authentication, department and designation management, attendance tracking, leave workflow, leave balances, payroll computation, payslips, employee self-service, manager dashboard, holiday calendar, performance notes, HR analytics, and role-based access control.

System Architecture

The system follows a layered MVC architecture. The presentation layer is a responsive HTML, CSS, and JavaScript dashboard. The Express API receives authenticated requests, applies validation and authorisation middleware, delegates business rules to controllers, and persists approved data through Mongoose models to MongoDB.

Architecture Flow

Layer	Components	Responsibility
Client	Responsive HR dashboard, employee self-service views	Collects user input and presents permitted dashboard data.
API and routing	Node.js, Express.js, REST endpoints	Maps requests to controllers and applies route protection.
Business layer	Controllers, services, workflow checks	Enforces rules such as no duplicate clock-in and manager-only leave decisions.
Data layer	Mongoose models, MongoDB collections	Stores structured records and reference relationships.
Cross cutting	JWT, bcrypt, validation, error handler	Protects data and provides consistent JSON responses.

Technology Stack

Area	Technology	Purpose
Backend	Node.js and Express.js	REST API and middleware pipeline
Database	MongoDB with Mongoose	Flexible document storage and relationship references
Security	JWT and bcrypt	Authenticated sessions and password hashing
Validation	express-validator or Joi	Server-side request validation
Frontend	HTML, CSS and JavaScript	Dashboard demonstration and API consumption
Testing	Postman	Endpoint verification and documented test cases

Security and Reliability Design

Passwords are never stored as plain text. Configuration values such as the MongoDB URI, JWT secret, and port are loaded from environment variables. Protected routes verify a token and role before business logic runs. A single error-handling middleware returns a stable response structure with success, message, errorCode, and an appropriate HTTP status code.

Functional Modules

The modules below define the functional baseline for the system. Each implementation should be demonstrated through a working UI action or a documented Postman request during evaluation.

No	Module	Core Behaviour
1	Employee onboarding and authentication	HR creates accounts; login produces a JWT for approved access.
2	Department and designation management	HR maintains departments, heads, and organisational job titles.
3	Daily attendance tracking	Employees clock in and out; duplicate or invalid sequences are rejected.
4	Leave request and approval	Employees request leave; managers approve or reject with remarks.
5	Leave balance management	Casual, sick, and earned leave balances are tracked per employee.
6	Payroll computation engine	Base pay, attendance, and approved deductions determine net pay.
7	Payslip generation records	Computed monthly payroll data is retained for employee payslip history.

No	Module	Core Behaviour
8	Employee self-service profile	Employees view and update limited profile details.
9	Manager team dashboard	Managers review their team attendance and pending leave items.
10	Holiday calendar management	HR defines holidays that influence attendance and payroll rules.
11	Performance note logging	Managers record periodic employee performance observations.
12	HR reports and analytics	Attendance and payroll cost reports are available by department.
13	Role-based access control	Employee, Manager, and HR Admin permission boundaries are enforced.

Database Design and Relationships

MongoDB collections are designed around the system’s read patterns. Shared and independently updated entities are referenced with ObjectIds; small attributes that always travel with their parent may be embedded. This avoids excessive joins while retaining a clear source of truth for cross-module records.

Collection	Key Fields	Relationship Design
users	name, email, passwordHash, role, departmentId, managerId	departmentId and managerId reference users/departments; email is unique.
attendance	employeeId, date, clockIn, clockOut, status	Many attendance records reference one employee.
leaveRequests	employeeId, type, fromDate, toDate, status, approverId	References requesting employee and approving manager.
leaveBalances	employeeId, casual, sick, earned	One balance document per employee; small leave counters are embedded.
payroll	employeeId, month, year, baseSalary, deductions, netPay	One employee can have many monthly payroll records.
departments	name, headId	References the user who heads the department.
holidays	date, description	Independent calendar records consumed by attendance and payroll.

Reference Diagram

users -> attendance (one to many) users -> leaveRequests (one to many) users -> payroll (one to many)

departments -> users (one to many) users -> departments via headId (one to optional one) users -> leaveBalances (one to one)

Indexes

Collection	Index	Reason
users	{ email: 1 } unique	Fast login lookup and prevents duplicate accounts.
attendance	{ employeeId: 1, date: 1 } unique	Fast employee history lookup and one attendance record per day.
leaveRequests	{ employeeId: 1, status: 1 }	Efficient leave history and pending status queries.
payroll	{ employeeId: 1, month: 1, year: 1 } unique	Prevents duplicate monthly payslips.
departments	{ name: 1 } unique	Avoids duplicate department definitions.

API Design and Workflow Rules

The API follows resource-oriented REST conventions. Controllers validate input, check token identity and role, apply workflow conditions, and return predictable JSON structures. The examples below form the core endpoint set; the full Postman collection should include every implemented route.

Method	Endpoint	Authorised Role	Purpose
POST	/api/auth/register	HR Admin	Create an employee account.
POST	/api/auth/login	Public	Authenticate and receive JWT.
POST	/api/attendance/clockin	Employee	Create today's open attendance record.
POST	/api/attendance/clockout	Employee	Close today's open attendance record.
POST	/api/leaves	Employee	Submit a leave request.
PUT	/api/leaves/:id/decision	Manager	Approve or reject a team request.
POST	/api/payroll/run	HR Admin	Calculate a monthly payroll run.
GET	/api/employees/:id/payslips	Employee or HR Admin	View payslip history with ownership check.
GET	/api/hr/reports/attendance	HR Admin	Read department attendance analytics.

Business Rule Examples

Workflow	Rule	Failure Response
Clock in	An employee may not create a second open attendance record for the same date.	409 CONFLICT with DUPLICATE_CLOCK_IN.
Clock out	Clock out requires an existing open record owned by the authenticated employee.	400 or 403 with a meaningful error code.
Leave request	Dates must be valid, non-overlapping, and within the available balance.	400 or 409 with VALIDATION_ERROR or LEAVE_BALANCE_EXCEEDED.
Leave decision	Only the designated manager or authorised HR role may decide a pending request.	403 FORBIDDEN or 409 INVALID_STATUS_TRANSITION.
Payroll	A payroll period cannot be finalised twice for the same employee and month.	409 PAYROLL_ALREADY_EXISTS.

User Interface and Dashboard

The accompanying PeopleFlow dashboard prototype offers a clean HR Administrator landing page. It prioritises the information needed at the beginning of a working day: organisation headcount, employees on leave, present staff, projected payroll cost, attendance distribution, pending approvals, weekly trend, and holiday context.

Dashboard Elements

Area	Information Presented	User Value
Summary cards	Total employees, present today, employees on leave, payroll cost	Immediate operational status without opening multiple records.
Attendance panel	Present, remote, on leave, and absent distribution	Supports same-day follow-up and manager visibility.
Leave requests	Employee, leave type, requested dates, number of days	Draws attention to approvals needing a manager decision.
Payroll panel	Estimated payout and processing date	Helps HR prepare the monthly payroll run.
Trend and calendar	Weekly attendance bars and holiday date	Provides context for changes in availability.
Add employee dialog	Name and department input with confirmation	Demonstrates a simple onboarding interaction.

Frontend Integration Plan

The static interface becomes data-driven by calling protected endpoints after login, storing the JWT safely for the active session, rendering API responses into cards and tables, and showing server-provided validation messages. Frontend checks improve usability, but all security and validation decisions remain on the Express server.

Testing and Validation Plan

Testing is performed through a Postman collection and a short UI walkthrough. Every endpoint should be tested for a valid request, invalid input, no token, wrong role, business-rule conflict, and a non-existent resource. This confirms that the backend is not merely performing CRUD but is enforcing the intended workflow.

Test Case	Example Action	Expected Result
Happy path	HR creates a valid employee account.	201 Created with a new user identifier.
Validation failure	Send leave request without a leave type.	400 Bad Request and VALIDATION_ERROR.
Authentication failure	Call payroll run without a bearer token.	401 Unauthorized.
Authorisation failure	Employee attempts leave decision endpoint.	403 Forbidden.
Business conflict	Employee attempts a second clock-in on the same day.	409 Conflict; no duplicate attendance document.
Not found	Query a valid-format identifier that has no record.	404 Not Found; server remains available.
Payroll accuracy	Run payroll with known base pay and approved deductions.	Net pay equals documented calculation and payslip is stored.

Consistent Error Response

All failed requests return JSON in the following form: success: false, message: a readable explanation, and errorCode: a stable machine-readable identifier. This helps the UI display clear feedback and helps developers diagnose errors without exposing secrets or stack traces.

Validation Checklist

Required fields, email format, password strength, MongoDB ObjectId format, leave dates, enums for roles and statuses, unique departmental names, and payroll period uniqueness should be validated before controller logic writes to the database.

Implementation Plan and Team Contribution

The project is organised as a 12-day sprint so that the foundation and core workflows are ready before reporting and submission work begins. Feature branches, frequent commits, and shared API contracts reduce merge conflicts and ensure every member understands the complete application.

Phase	Dates	Primary Deliverables
Foundation	1 to 2 September	Requirement analysis, team setup, repository, schema design, environment configuration.
Core Build	3 to 6 September	Authentication, employee and department CRUD, attendance routes, initial frontend.
Workflow and Testing	7 to 9 September	Leave, balances, payroll, approval rules, Postman collection, error cases.
Wrap Up	10 to 11 September	Reports, analytics, README, PPT, screenshots, full rehearsal.
Submission	12 September	Final GitHub push and one LMS PDF submission before 11:59 PM.

Member	Suggested Ownership	Shared Responsibility
Member 1	Authentication, employee setup, departments, attendance, leave requests	Review schema and explain all modules in viva.
Member 2	Leave balances, payroll engine, payslips, employee self-service	Review Postman collection and business rules.
Member 3	Manager dashboard, holidays, performance notes, analytics, RBAC	Review documentation and test error paths.
Member 4 if applicable	Schema design, validation, test collection, README and PPT consolidation	Support integration and final submission checks.

Repository Structure and Deployment Readiness

A predictable project structure keeps route definitions, business logic, data models, and middleware independently testable. Environment files and runtime dependencies must not be committed to GitHub.

Path	Purpose
config/db.js	MongoDB connection and configuration loader.
models/	Mongoose schemas for users, attendance, leave requests, balances, payroll, departments, holidays, and notes.
routes/	Express route definitions grouped by resource.
controllers/	Business logic, permissions, and workflow transitions.
middleware/	JWT authentication, role checks, request validation, and central error handler.
utils/	Token generation, pagination, payroll helpers, and shared constants.
.env.example	Safe sample configuration names with no real credentials.
README.md	Project overview, team details, setup instructions, modules, API reference, and data design rationale.
postman/	Exported collection and environment documentation.

GitHub Hygiene

The repository should be public or shared with the faculty coordinator, include meaningful commits from multiple members, and exclude node_modules and .env through .gitignore. Each contribution should be pushed frequently rather than only at the deadline.

Known Constraints

The academic scope assumes one currency, one locale, and one time zone. External payment, SMS, and email services may be mocked. The core evaluation priority remains backend design, MongoDB modelling, API correctness, and demonstrable workflow logic.

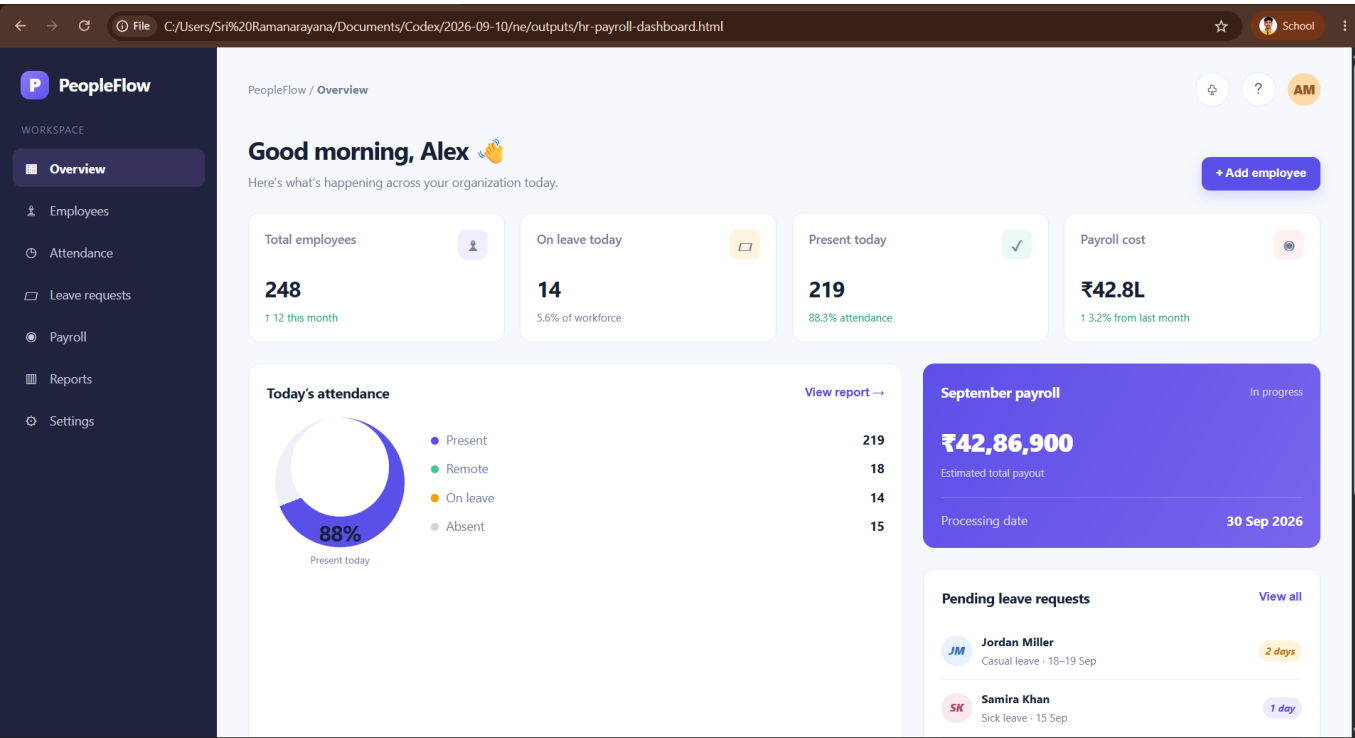
Conclusion and Submission Checklist

The Corporate HR and Payroll Management System provides a coherent backend project that connects employee administration with operational workflows and payroll records. Its value lies in clear role boundaries, deliberate MongoDB relationships, validated APIs, and workflow rules that prevent invalid actions. The dashboard prototype strengthens the demonstration by making the system easier to navigate and explain.

Final Checklist

Item	Completion Before Submission
Team details page	Replace all placeholders and confirm the first report page is complete.
GitHub repository	Push source code, README, .gitignore, .env.example, Postman collection, and meaningful commits.
Working functionality	Demonstrate all thirteen required modules and test at least the specified error paths.
Security and validation	Confirm JWT, bcrypt, server-side validators, and central error middleware are active.
Report and PPT	Add repository link, final screenshots, and confirm every member can explain each module.
LMS upload	Export as ProjectCode_TeamNo_Section.pdf and submit by 11:59 PM on 12 September 2026.

Output Screenshots:



Add an employee

Create a new employee profile to begin onboarding.

Full name

e.g. Priya Sharma

Department

Engineering

Cancel

Create employee

Add an employee

Create a new employee profile to begin onboarding.

Full name

SRI RAM

Department

Engineering

Engineering

Human Resources

Finance

Marketing

